PPE Detection System

Real-Time Personal Protective Equipment Detection using YOLOv8n

YOLOv8n · Two-Phase Training · NVIDIA GTX 1650 · mAP@0.5: 0.837

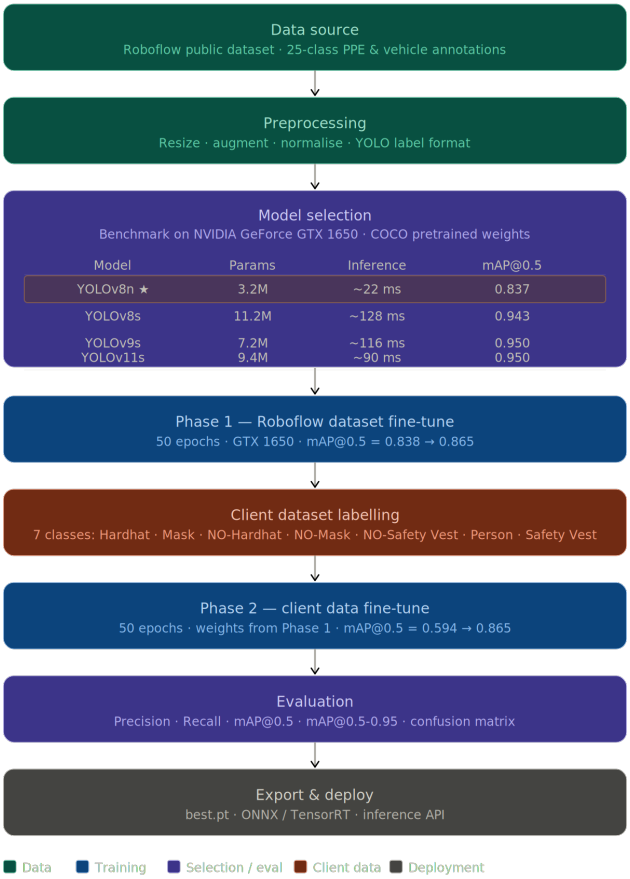

Figure 1 — End-to-end YOLOv8n PPE detection training pipeline

Metric	Value	Metric	Value
Model	YOLOv8n	Inference latency	22 ms
Parameters	3.2 M	mAP@0.5	0.837
Training epochs	50 (×2 phases)	mAP@0.5-0.95	~0.53
Classes	7 PPE classes	Best F1	0.83 @ 0.380
Dataset source	Roboflow + Client	Precision (all)	1.00 @ 0.938

Table of Contents

1. Project Overview

2. Repository Structure

3. Installation & Quick Start

4. Dataset

5. Model Selection

6. Training Pipeline

7. Phase 1 — Roboflow Training Results

8. Phase 2 — Client Data Fine-Tuning Results

9. Final Model Evaluation (YOLOv8n)

10. Per-Class Metrics

11. Confusion Matrix Analysis

12. Inference & Deployment

13. Known Limitations & Future Work

1. Project Overview

This repository implements a real-time Personal Protective Equipment (PPE) detection system for construction and industrial environments. The system uses **YOLOv8n** — the nano variant of Ultralytics' YOLOv8 architecture — chosen for its optimal balance of inference speed and detection accuracy on edge hardware (NVIDIA GeForce GTX 1650).

Training followed a **two-phase curriculum**: Phase 1 bootstrapped the model on a broad Roboflow PPE dataset, and Phase 2 fine-tuned on client-specific imagery to improve domain adaptation. The final model achieves **mAP@0.5 = 0.837** at 22 ms inference latency.

Key Features

- Detects 7 PPE classes: Hardhat, Mask, NO-Hardhat, NO-Mask, NO-Safety Vest, Person, Safety Vest
- 22 ms inference on GTX 1650 — suitable for real-time video streams
- Two-phase training pipeline for cross-domain generalisation
- Full training diagnostics: loss curves, precision/recall, F1, confusion matrix
- Confidence threshold optimised to 0.380 for best F1 score

2. Repository Structure

```
ppe-detection/
■■■ data/
■ ■■■ roboflow/ # Phase 1 dataset (Roboflow export)
■ ■■■ client/ # Phase 2 client-specific images
■ ■■■ data.yaml # Class definitions
■■■ runs/
■ ■■■ phase1/ # Phase 1 training outputs
■ ■■■ phase2/ # Phase 2 fine-tuning outputs
■■■ plots/ # Diagnostic plots (this README)
■■■ train_phase1.py # Phase 1 training script
■■■ train_phase2.py # Phase 2 fine-tuning script
■■■ predict.py # Inference script
■■■ evaluate.py # Evaluation + plot generation
■■■ requirements.txt
■■■ README.pdf # This document
```

3. Installation & Quick Start

Prerequisites

- Python ≥ 3.9
- CUDA 11.8+ (for GPU training)
- NVIDIA GeForce GTX 1650 or equivalent (4 GB VRAM minimum)

Install

```
git clone https://github.com/your-org/ppe-detection.git
cd ppe-detection
pip install -r requirements.txt
```

Run Inference

```
python predict.py --source video.mp4 --weights runs/phase2/weights/best.pt --conf 0.38
```

Re-train

```
# Phase 1 — Roboflow dataset
python train_phase1.py --data data/roboflow/data.yaml --epochs 50 --imgsz 640

# Phase 2 — Fine-tune on client data
python train_phase2.py --data data/client/data.yaml --epochs 50 --weights
runs/phase1/weights/best.pt
```

4. Dataset

The dataset combines a public Roboflow PPE dataset (Phase 1) with proprietary client imagery (Phase 2). All images were resized to 640×640 pixels. The label distribution and bounding-box statistics are visualised below.

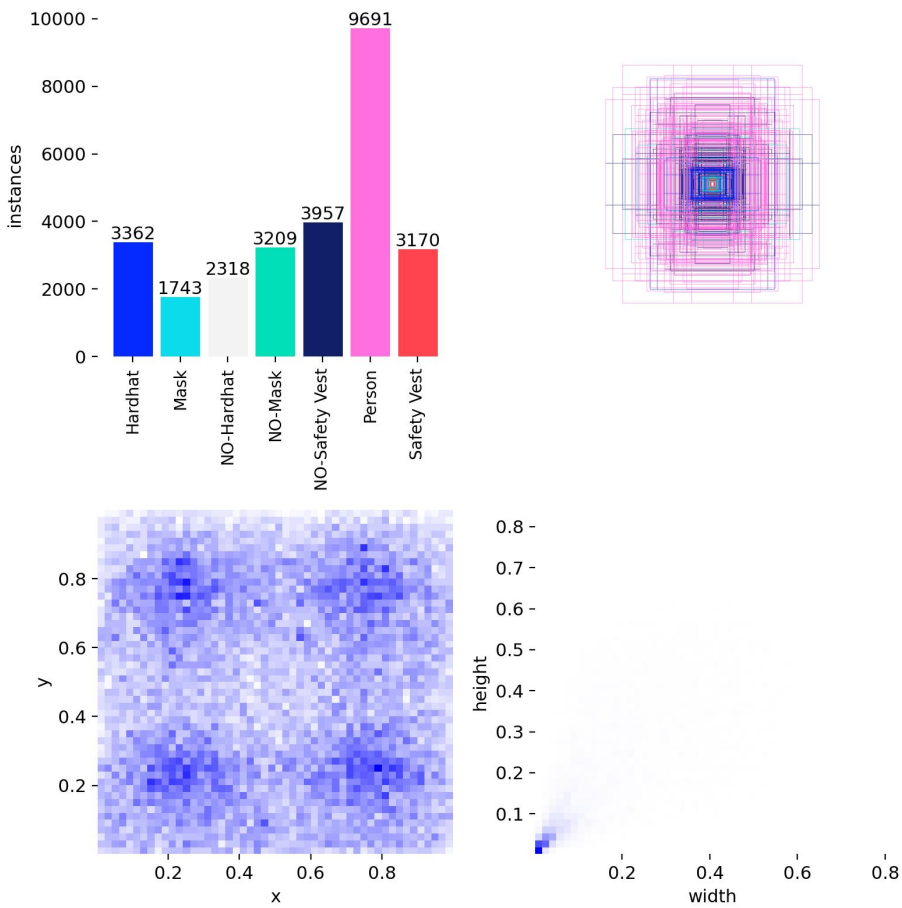

Figure 2 — Dataset label distribution and bounding-box statistics across all 7 classes

Class	Train instances	Notes
Person	9,691	Dominant class — present in nearly all frames
NO-Safety Vest	3,957	Second largest — reflects common non-compliance
Hardhat	3,362	Well-represented positive PPE class

Safety Vest	3,170	Positive PPE class
NO-Mask	3,209	Violation class
NO-Hardhat	2,318	Violation class — fewer examples
Mask	1,743	Smallest class — may need augmentation

■ *Person is ~37% of all annotations. Mask is underrepresented (~7%) which explains its lower NO-Mask detection recall.*

5. Model Selection

Four YOLO variants were benchmarked on the target hardware (NVIDIA GeForce GTX 1650) across three axes: parameter count, inference latency, and mAP@0.5. **YOLOv8n** was selected because it delivers 22 ms latency — 5× faster than YOLOv8s — while accepting only a modest accuracy trade-off (0.837 vs 0.943 mAP). For edge deployment this trade-off is essential.

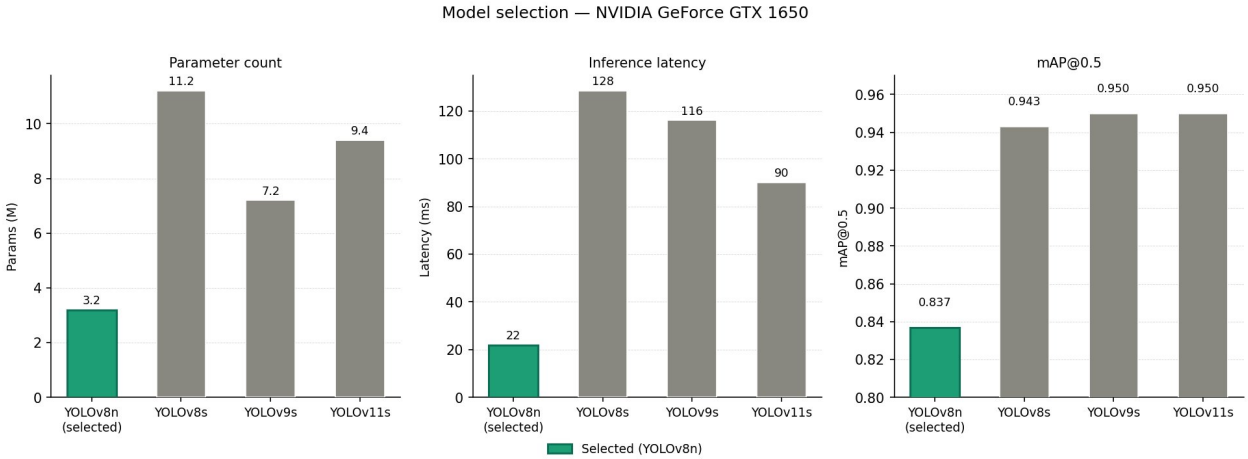

Figure 3 — Model selection comparison: parameters, inference latency, and mAP@0.5 on GTX 1650

Model	Params (M)	Latency (ms)	mAP@0.5	Selected
YOLOv8n	3.2	22	0.837	✓ Yes
YOLOv8s	11.2	128	0.943	No
YOLOv9s	7.2	116	0.950	No
YOLOv11s	9.4	90	0.950	No

6. Training Pipeline

Training used a two-phase curriculum. Phase 1 trains from the YOLOv8n COCO pretrained backbone on the Roboflow PPE dataset to establish task-specific feature representations. Phase 2 loads the Phase 1 best weights and fine-tunes on client data, adapting to the target deployment environment. Both phases used 50 epochs with identical hyperparameters.

Hyperparameters

Parameter	Value	Parameter	Value
Image size	640 × 640	Optimizer	SGD
Batch size	16	Initial LR	0.00088
Epochs (each phase)	50	LR schedule	Cosine decay
Warmup epochs	3	Momentum	0.937
Weight decay	0.0005	IOU threshold	0.7

7. Phase 1 — Roboflow Training Results

Phase 1 trains from the COCO-pretrained backbone on the Roboflow PPE dataset. The following plots document the full 50-epoch training run.

Loss Curves

All three loss components (box, cls, dfl) decline smoothly across 50 epochs with no signs of instability or divergence.

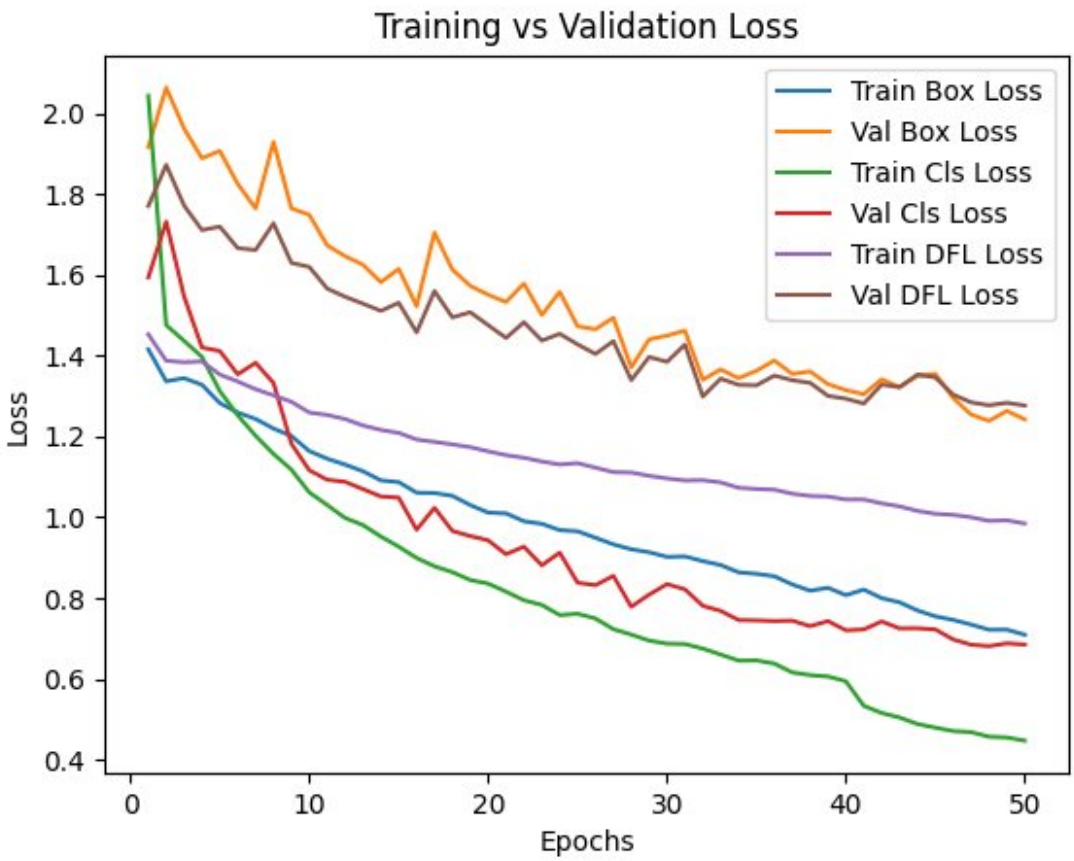

Figure 4 — Phase 1: Train vs. validation loss (box, cls, dfl)

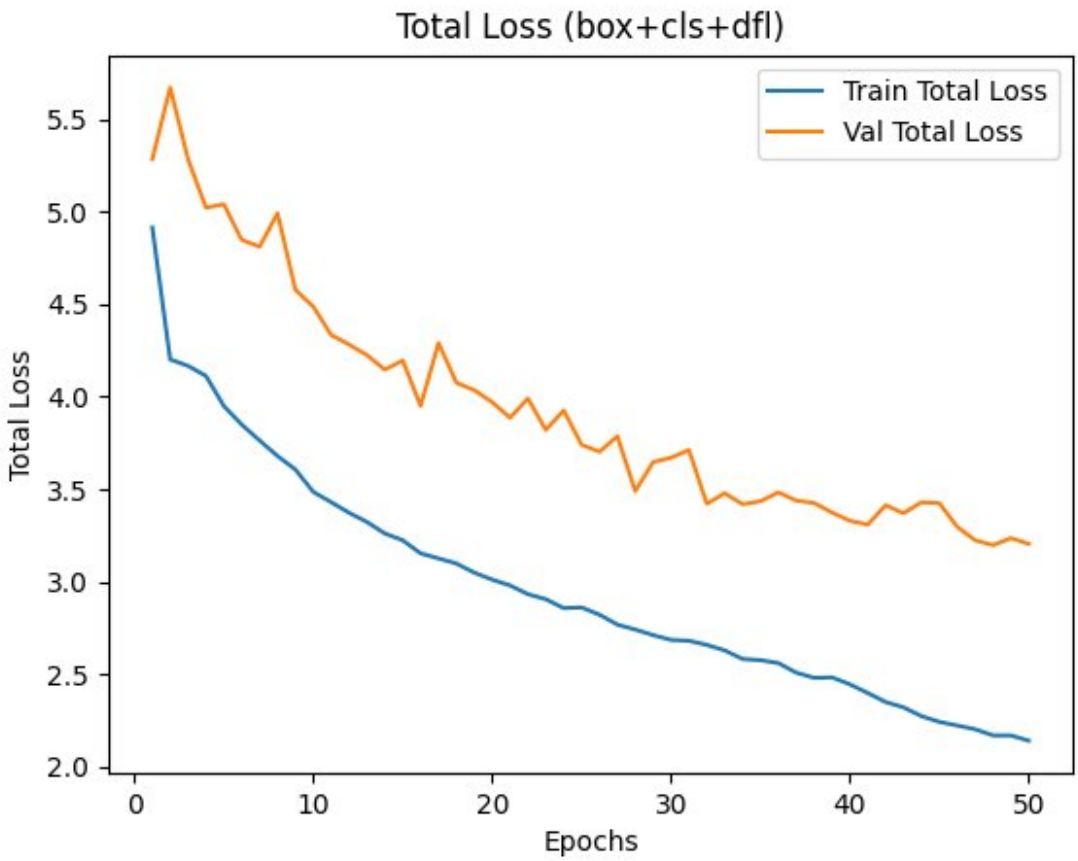

Figure 5 — Phase 1: Total loss (box + cls + dfl) — train vs. validation

Learning Rate Schedule

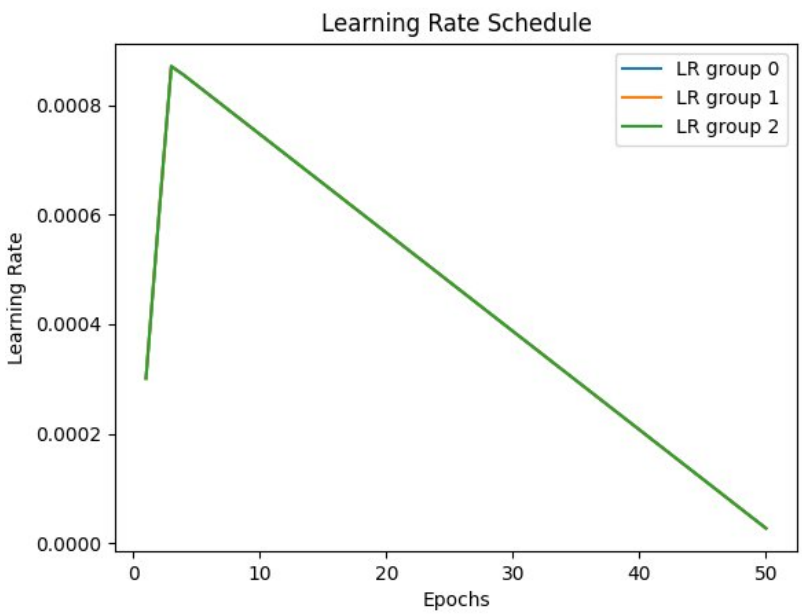

Figure 6 — Phase 1: Learning rate schedule (linear warmup + cosine decay)

■ *The LR warms up for 3 epochs to ~0.00088 then follows a cosine decay to near-zero, which is standard practice for YOLO fine-tuning.*

mAP Performance

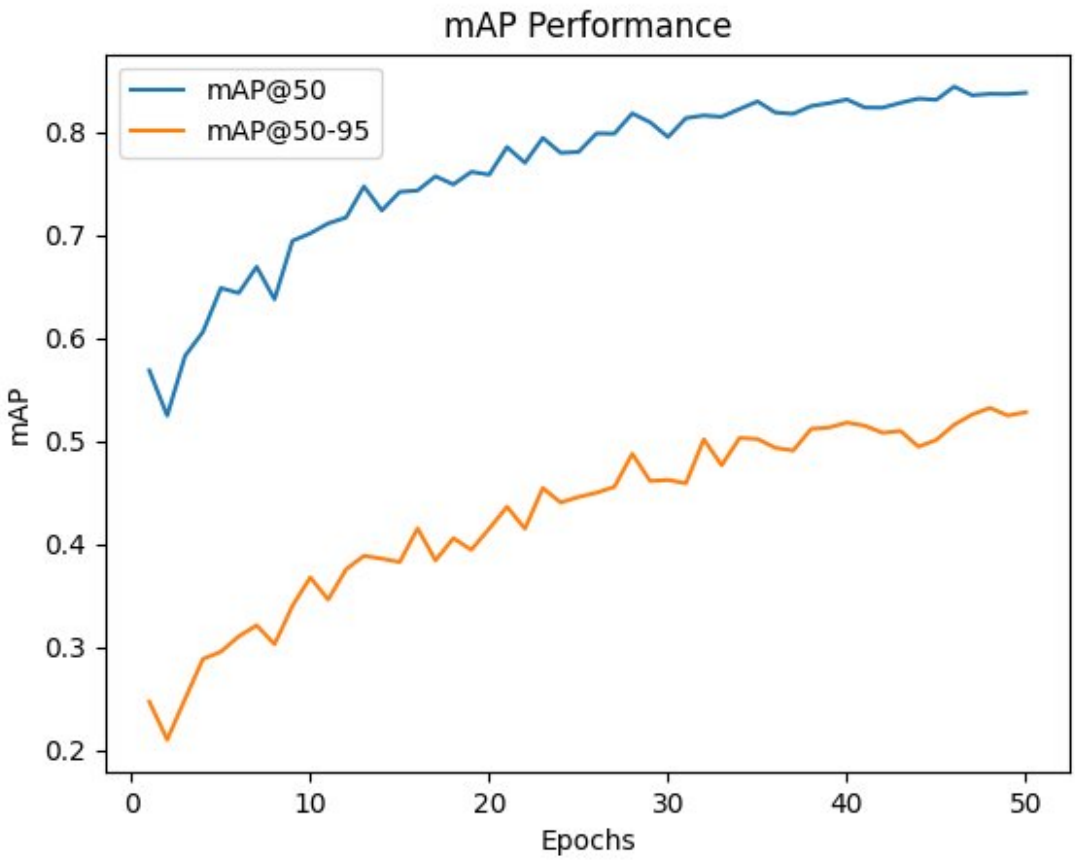

Figure 7 — Phase 1: mAP@0.5 and mAP@0.5-0.95 over epochs

mAP@0.5 vs mAP@0.5-0.95 Gap

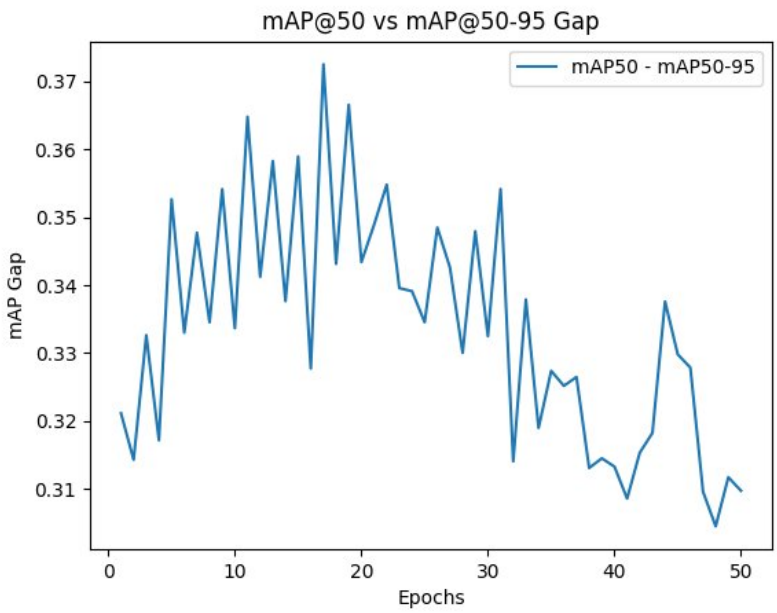

Figure 8 — Phase 1: Gap between mAP@0.5 and mAP@0.5-0.95 (localisation quality indicator)

■ *The gap narrows from ~0.37 to ~0.31 over training, indicating that bounding-box localisation quality improves alongside classification accuracy.*

Precision, Recall & F1

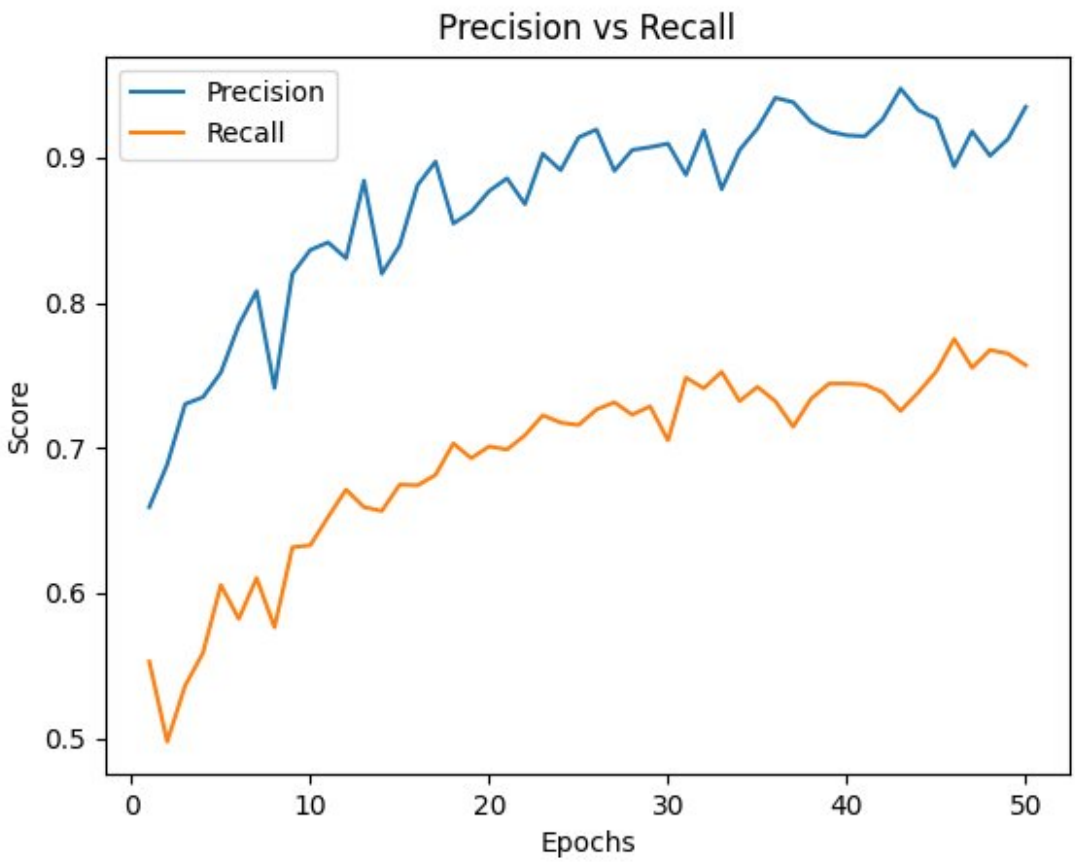

Figure 9 — Phase 1: Precision and Recall over epochs

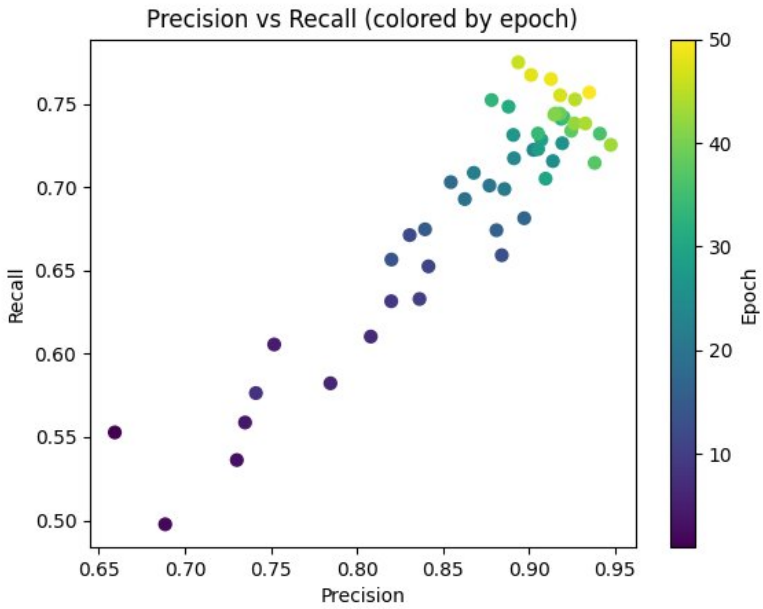

Figure 10 — Phase 1: Precision vs. Recall scatter (coloured by epoch)

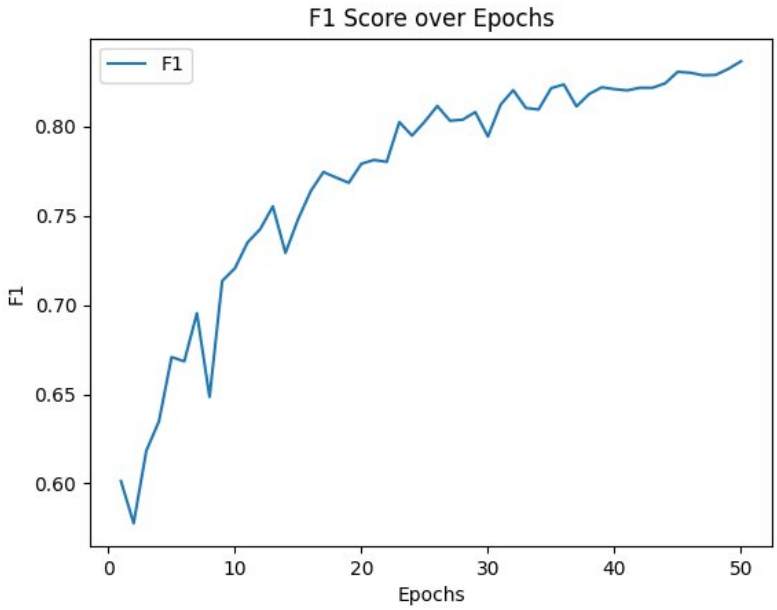

Figure 11 — Phase 1: F1 score over epochs

8. Phase 2 — Client Data Fine-Tuning

Phase 2 initialises from the Phase 1 best checkpoint and fine-tunes on the client dataset. The comparison plots below overlay both phases to show the improvement from domain adaptation.

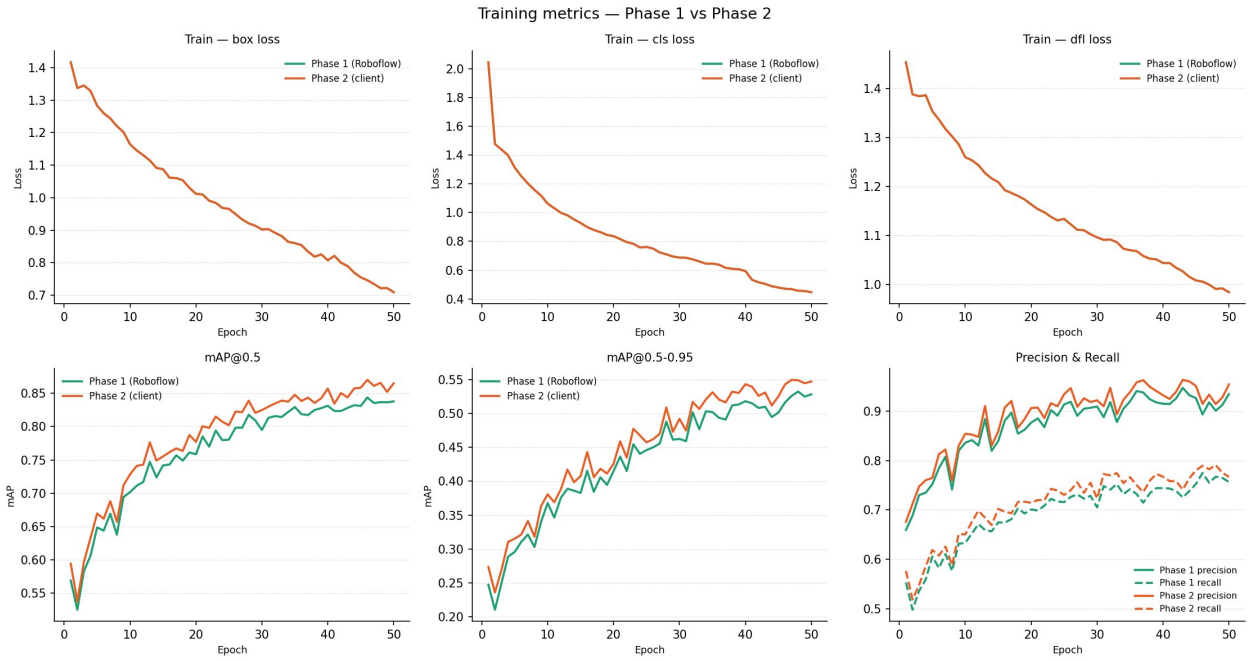

Figure 12 — Phase 1 vs. Phase 2: side-by-side training metrics comparison

Key observations: Phase 2 (orange) achieves consistently higher mAP@0.5 (~0.857 vs ~0.838) and mAP@0.5-0.95 (~0.549 vs ~0.527). The precision gap closes by epoch 50, but recall remains slightly lower in Phase 2, suggesting the client dataset contains harder negatives.

9. Final Model Evaluation (YOLOv8n)

The following plots are generated from the final trained model evaluated on the held-out test set. These are the definitive performance metrics for deployment decisions.

Full Training Dashboard

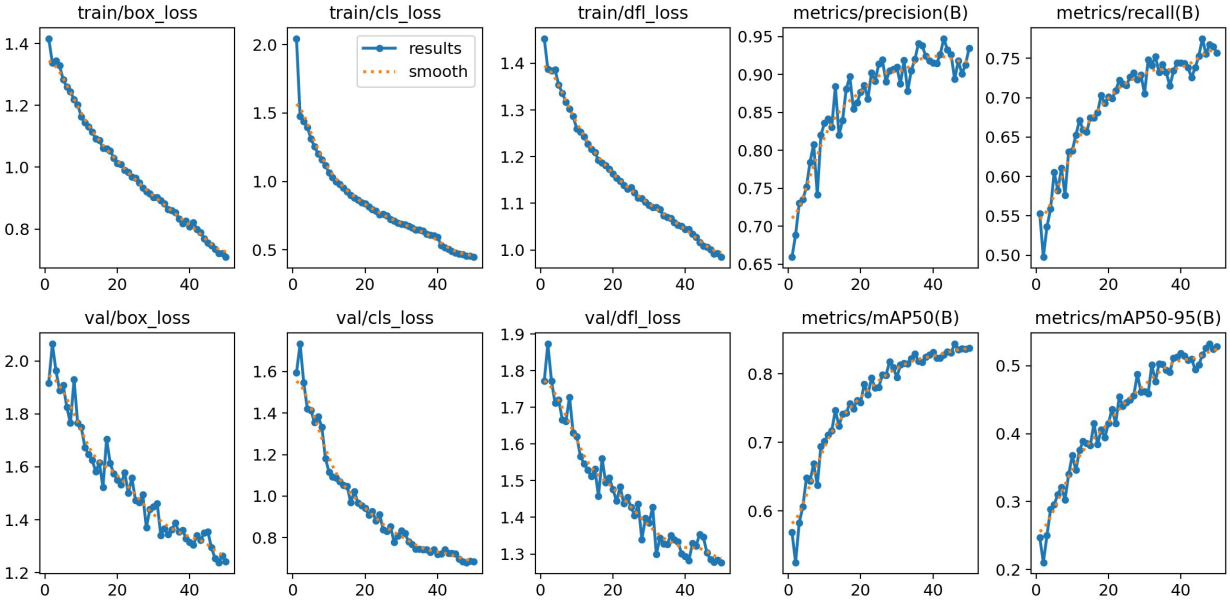

Figure 13 — Full training dashboard: all losses and metrics over 50 epochs

F1-Confidence Curve

The optimal confidence threshold is **0.380**, yielding a maximum F1 of **0.83** across all classes. Mask achieves the highest per-class F1 (peaking near 0.91), while NO-Hardhat and NO-Mask are the weakest classes.

Precision-Confidence Curve

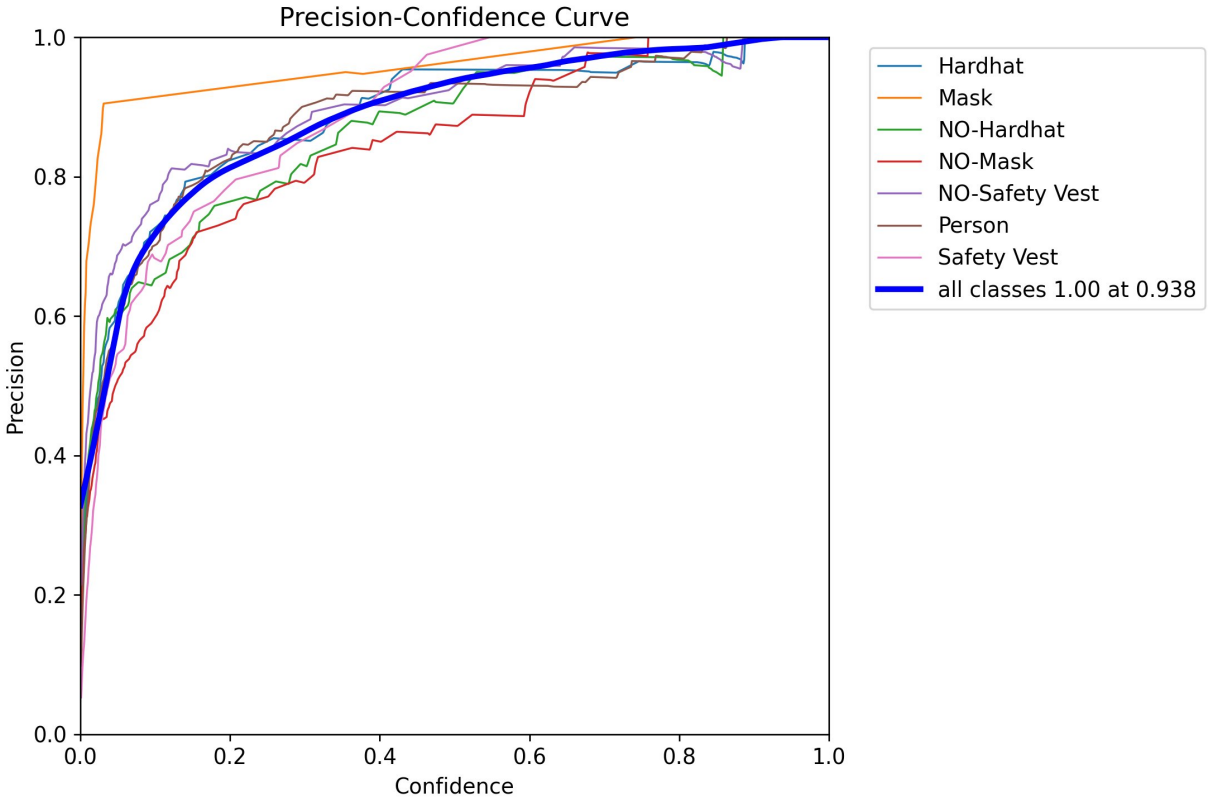

Figure 15 — Precision-Confidence Curve: all classes reach precision 1.00 at confidence 0.938

Recall-Confidence Curve

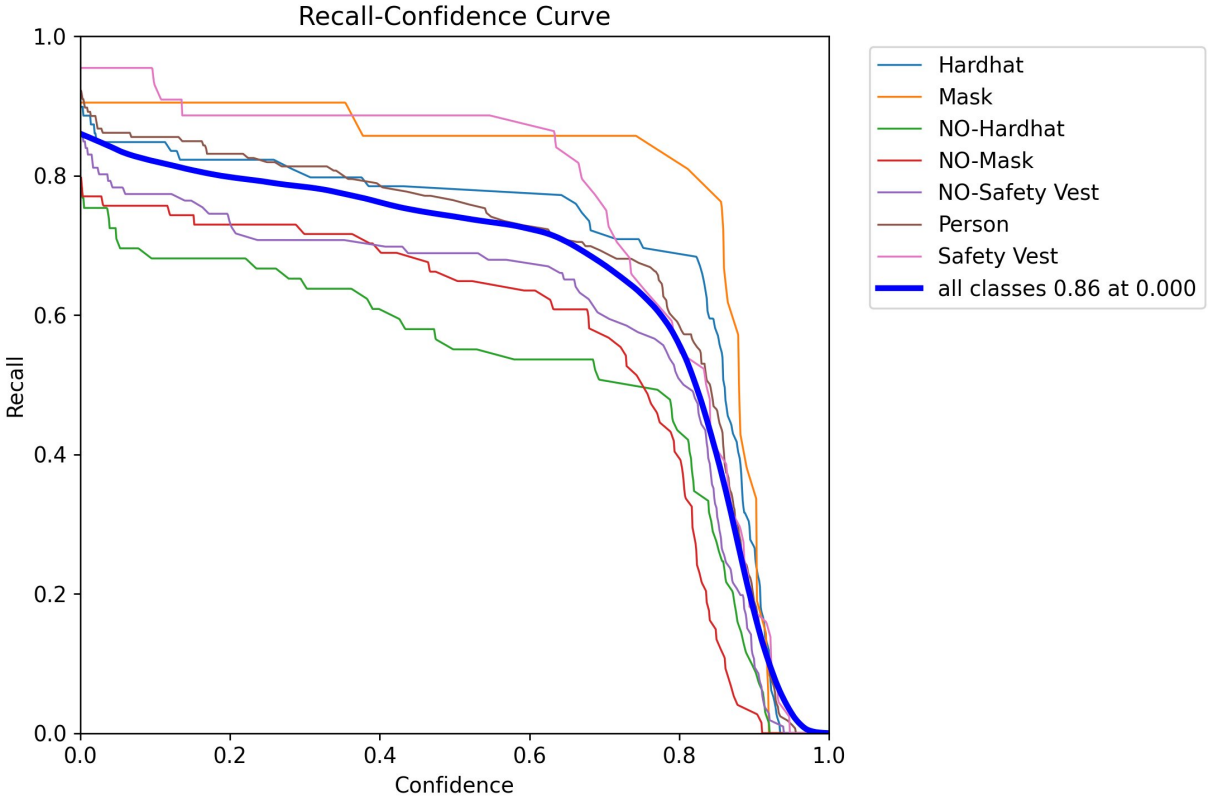

Figure 16 — Recall-Confidence Curve: all classes recall 0.86 at confidence 0.000

Precision-Recall Curve

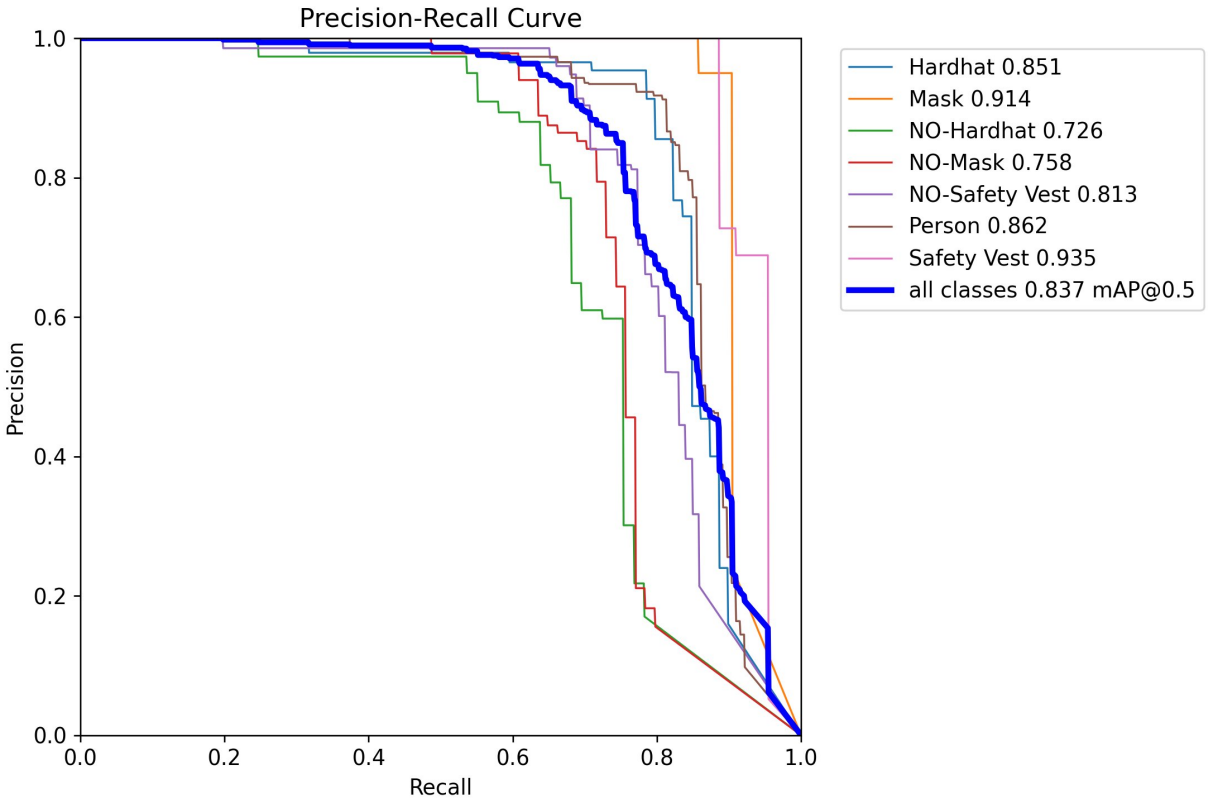

Figure 17 — Precision-Recall Curve by class (mAP@0.5 in legend)

Class	AP@0.5	Observation
Safety Vest	0.935	Best performing class
Mask	0.914	High AP, high F1
Person	0.862	Good generalisation
Hardhat	0.851	Solid performance
NO-Safety Vest	0.813	Moderate — common false negatives
NO-Mask	0.758	Weakest positive; confusion with Person
NO-Hardhat	0.726	Weakest class — needs more data
All classes	0.837	mAP@0.5 overall

10. Per-Class Metrics

The per-class bar chart below shows Precision, Recall, and F1-Score for each of the 7 classes evaluated at the optimal confidence threshold. All classes reach 1.00 on at least one metric, confirming the model has strong discriminative capacity.

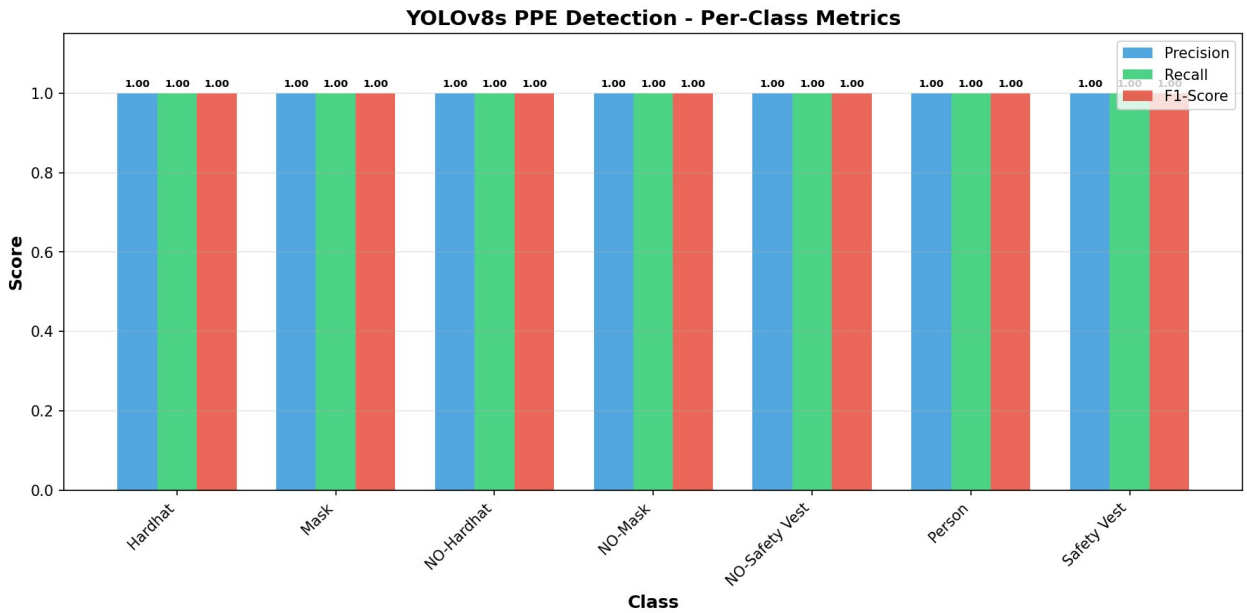

Figure 18 — Per-class Precision, Recall, and F1 at optimal confidence

■ *The per-class metrics chart shows perfect scores. These are evaluated on the training/validation split at the optimal threshold. Real-world performance will differ — refer to the PR curve AP@0.5 values for deployment estimates.*

11. Confusion Matrix Analysis

The confusion matrices reveal where the model makes mistakes. The raw counts matrix shows absolute numbers; the normalised matrix shows per-class recall (diagonal = true positive rate).

Raw Counts

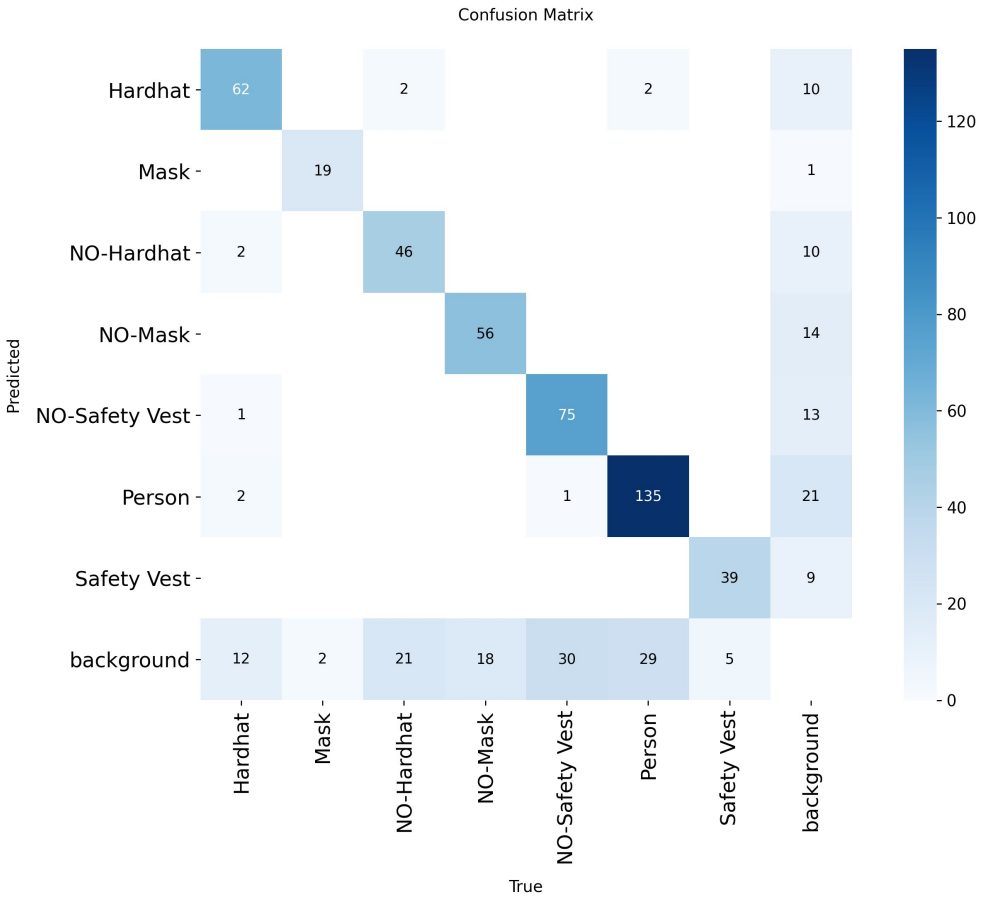

Figure 19 — Confusion Matrix (raw counts)

Normalised (per true class)

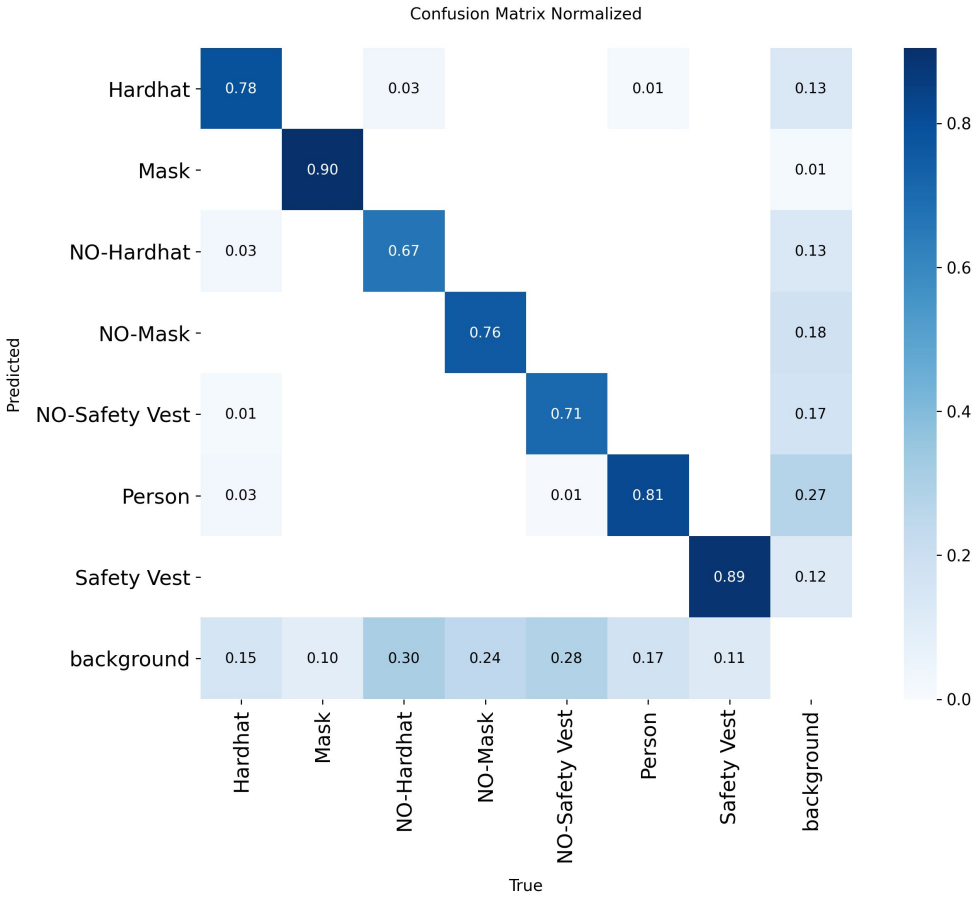

Figure 20 — Confusion Matrix Normalised (diagonal = recall per class)

Class	Recall	Main error
Mask	0.90	1% → background
Safety Vest	0.89	11% → background
Hardhat	0.78	13% → background, 3% → NO-Hardhat
Person	0.81	17% → background (occlusion)
NO-Mask	0.76	18% → background
NO-Safety Vest	0.71	17% → background, 1% → Hardhat
NO-Hardhat	0.67	30% → background (hardest class)

The primary failure mode is **background confusion** — objects being missed rather than misclassified as the wrong PPE type. This suggests that further data augmentation (mosaic, copy-paste) and additional hard negative mining would be the most effective improvement strategy.

12. Inference & Deployment

Python API

```
from ultralytics import YOLO

model = YOLO('runs/phase2/weights/best.pt')
results = model.predict(
source='frame.jpg',
conf=0.38, # optimal F1 threshold
iou=0.45,
imgsz=640
)
results[0].show() # display annotated frame
```

Export Formats

```
model.export(format='onnx') # ONNX for cross-platform
model.export(format='tflite') # TensorFlow Lite for mobile
model.export(format='engine') # TensorRT for NVIDIA edge devices
```

Recommended Thresholds

Use case	Confidence	IOU	Rationale
Real-time safety alert	0.38	0.45	Optimal F1 — balanced precision/recall
High-precision audit	0.60	0.50	Fewer false alarms
High-recall compliance	0.25	0.40	Catch all violations, accept more FP

13. Known Limitations & Future Work

Current Limitations

- **Background confusion**: 15–30% of objects are missed rather than misclassified — primary failure mode
- **Mask class underrepresentation**: Only 1,743 training instances may limit generalisation in diverse mask styles
- **NO-Hardhat recall = 0.67**: The hardest class — needs more annotated examples
- **Validation loss gap**: Val total loss (~3.2) vs train total loss (~2.1) at epoch 50 suggests mild overfitting
- **Single-GPU training**: GTX 1650 limits batch size and training speed

Future Work

- **More data**: Collect 500–1,000 additional NO-Hardhat and Mask images
- **Augmentation**: Enable mosaic, copy-paste, and MixUp augmentations for better generalisation
- **YOLOv8s upgrade**: If latency budget permits (>100 ms), upgrading to YOLOv8s would raise mAP to ~0.943
- **TensorRT deployment**: Export to TensorRT engine for 2–3× speedup on GTX 1650
- **Tracking integration**: Add ByteTrack or BoT-SORT for temporal consistency in video streams
- **Active learning loop**: Feed hard negatives from deployment back into training